

● LEXY

ENGINEERING DOCUMENTATION

DOCUMENT 02 / 09

Architecture Overview

AI Hiring Platform · Intelligence-First

Generated **June 14, 2026** · Confidential — internal use

Lexy — Architecture Overview

Engineering documentation set · Doc 2 of 9 How the pieces fit together. For *why* they are shaped this way, see Doc 4.

■ 1. One-paragraph mental model

A **pnpm monorepo** holds several deployable React apps (`artifacts/*`) and shared TypeScript libraries (`lib/*`). One Express backend (`@workspace/api-server`) owns all business logic, the AI agents, the AI job queue, and ~16 background schedulers. State lives in **PostgreSQL** accessed through **Drizzle ORM**, and tenant isolation is enforced *in the database* with **Row-Level Security (RLS)**. The frontend talks to the backend over a JSON API mounted at `/api`, using a generated React Query client plus a thin `apiFetch` wrapper that attaches a bearer token.



† **On the bearer token:** it carries a JWT-style payload (`{ userId, tenantId, role, region }`) but is **signed and verified with HMAC-SHA256 using a shared server secret** — it is *not* issued or validated through a standard JWT library, and there is no RS256/public-key verification or external identity provider. The server is both issuer and verifier.

`withTenantContext` and `resolveUser` treat the HMAC-verified payload as the source of truth (no DB lookup per request). Practically: trust it like a signed session token, not like a third-party OIDC/JWT you'd validate against a JWKS endpoint.

■ 2. The monorepo at a glance

Deployable apps — `artifacts/*`

Package	Role
<code>@workspace/api-server</code>	Express backend: routes, agents, intelligence engine, schedulers, AI job worker. The center of gravity.
<code>@workspace/lexy</code>	Main hiring platform SPA (recruiter + candidate portal).
<code>@workspace/lexy-site</code>	Public marketing website.
<code>@workspace/lexy-brochure</code> , <code>@workspace/lexy-candidate-brochure</code>	Slide/presentation surfaces.
<code>@workspace/mockup-sandbox</code>	Isolated component preview server.

Shared libraries — `lib/*`

Package	Role
<code>@workspace/db</code>	Drizzle schema, connection pooling, and the RLS-bound request proxy (the heart of tenant isolation).
<code>@workspace/api-zod</code>	Shared Zod schemas — the single source of request/response validation truth.
<code>@workspace/api-spec</code>	API spec + Orval codegen config.
<code>@workspace/api-client-react</code>	Generated React Query hooks consumed by the frontend.
<code>@workspace/integrations-openai-*</code>	Server + client OpenAI integration wrappers.
<code>@workspace/object-storage-web</code>	Web object-storage utilities (Uppy).

■ 3. Backend anatomy (`artifacts/api-server`)

Four cooperating subsystems live in one process (plus a separable worker):

1. **HTTP routes** (`src/routes/*`) — thin handlers grouped by domain (auth, sourcing, candidates, jobs, pipeline, interviews, outreach, intelligence, tenants, billing, audit, ...). They validate with Zod, call into libs, and never own business logic that needs to be shared.
2. **Agents + orchestrator** (`src/lib/agents/*`) — 10 specialized agents (icp, sourcing, screening, verification, outreach, interview, scheduling, anti-ghosting, proctoring, analytics) coordinated by `orchestrator.ts`. They run in three modes: synchronous (`triggerAgent`), as a chained **pipeline** (`runPipeline` , where one agent's output feeds the next), or via the async **AI job queue**.
3. **Intelligence engine** (`src/lib/intelligence.ts` + `learned-scoring.ts` , `global-prior.ts` , `backtest.ts` , `fairness.ts`) — turns agent signals into a composite score (Fit / Quality / Trust / Conversion → Hire Probability) plus a `next_best_action` , written to `candidate_job_intelligence` . This is the product's core; Doc 4 covers it in depth.
4. **Schedulers + AI job worker** — ~16 interval schedulers (outreach, anti-ghosting, trial expiry, subscription lifecycle, digests, platform recs, re-engagement, STT alerts/retention, LinkedIn monitor, ...) run inside the API process under a single scheduler-leader. Long-running / non-blocking work is instead pushed onto the `ai_jobs` table and drained by a worker loop using `FOR UPDATE SKIP LOCKED` for safe horizontal scaling.

■ 4. Data layer & tenant isolation (the load-bearing decision)

- **PostgreSQL + Drizzle ORM**, schema owned by `@workspace/db` .
- Every authenticated `/api/*` request passes through `withTenantContext` , which acquires a connection, `SET ROLE lexy_app` , and sets two GUCs: `app.current_tenant_id` and `app.allowed_tenant_ids` .
- **RLS policies** on key tables (`candidates` , `applications` , `interview_sessions` , ...) read those GUCs, so isolation is enforced by the database, not by remembering to add a `WHERE tenant_id = ...` in every query.
- `getAllowedTenantIds(user)` resolves a user's **subtree** via a recursive CTE: a parent tenant sees all descendants; siblings can't see each other; `platform_admin` returns `null` (sees everything).
- Candidates live in a **shared platform pool** (`candidates`) and are linked to tenants through `sourced_candidates` / `applications` . "Platform pool vs tenant candidate" is a recurring distinction — see Doc 4 and the memory topic files.

■ 5. AI / external service surface

Capability	Provider	Notes
LLM reasoning (ICP, screening, interview, scoring)	OpenAI (via integration proxy)	All candidate scorers wrapped with fairness directive + PII redaction.
Speech-to-text (interviews)	Azure Speech (+ Whisper fallback)	Pooled accounts + admission control for concurrent load.
Text-to-speech (interviews)	Azure TTS (separate account)	Watchdog ensures TTS always settles before mic opens.
Email send/receive	AWS SES + inbound parse	Outreach + auto-reply + recruiter inbox.
Object storage	AWS S3	Resumes, interview recordings (resumable multipart).
External sourcing	PDL, SERP, GitHub	Behind a pluggable provider adapter layer with per-kind timeouts.

■ 6. Frontend ↔ backend contract

- **Generated client:** `@workspace/api-client-react` (Orval → React Query hooks from the API spec) is the preferred path.
- **Custom wrapper:** `artifacts/lexy/src/lib/api.ts` exposes `apiFetch`, which attaches the bearer token from `localStorage` and prefixes the API base.
- **Conventions worth knowing** (see memory): there is *no* default queryFn — every `useQuery` supplies its own; base-URL variable name differs by scope (`BASE` vs `BASE_URL`); a null score renders as `—`, never `0%`.

■ 7. Deployment shape (summary — full detail in `DEPLOY.md`)

- **Backend** → a single Node.js **Docker container**. There is no committed production orchestrator yet: `DEPLOY.md` documents **two interchangeable paths** for running that container on AWS — **Elastic Beanstalk** (Option A, simpler) or **ECS Fargate** (Option B). Either runs the same image; pick one per environment. If you're about to debug a live issue, confirm which one the environment you're touching actually uses before assuming — they have different log locations and deploy mechanics.
- **Frontends** → static build to S3 behind CloudFront.
- Typical DNS split: `yourdomain.com` (site), `app.yourdomain.com` (platform), `api.yourdomain.com` (backend). The API server can also serve the built frontend if `FRONTEND_DIST_PATH` is set.

- On Replit (dev): each artifact is a workflow bound to `PORT` ; preview is a proxied iframe with path-based routing.